

Java Assignment 7

Marks: 30

Question 1: LCS with String Handling

[10 Marks]

Write a Java program LCSAnalyzer that finds the **Longest Common Subsequence (LCS)** of two sentences entered by the user and then analyses the result using different String concepts.

Your program must perform the following tasks:

- Input and validation:** Read two sentences using Scanner. If either sentence is empty or contains only spaces, print an error message and stop.
- Pre-processing:** Write a method that trims the text, converts it to lower case, removes every character other than letters, digits and spaces (use `replaceAll()`), and collapses multiple spaces into one.
- Character-level LCS:** Use dynamic programming (2-D table) with `charAt()` to find the LCS length, then back-track through the table and rebuild the LCS using `StringBuilder`.
- Similarity:** Print the LCS, its length and the similarity percentage = $(2 \times \text{LCS length}) / (\text{length1} + \text{length2}) \times 100$, formatted to 2 decimal places using `String.format()`.
- String analysis of the LCS:** Check whether the LCS is a palindrome (use `reverse()` and `equals()`), count the vowels and consonants using `indexOf()` / `Character.isLetter()`, and display the index of its first letter in both strings using `indexOf()` and `substring()`.
- Word-level LCS:** Split both sentences into words using `split()` and find the longest common sequence of words (compare with `equals()`). Print the result using `String.join()`.

Sample Input / Output:

```
Enter first sentence : The quick brown fox, jumps!
Enter second sentence: A quick brown dog jumped

Normalised 1: the quick brown fox jumps
Normalised 2: a quick brown dog jumped
LCS (characters) : " quick brown o jump"
LCS length      : 19
Similarity      : 77.55%
Palindrome?    : No
Vowels         : 5
Consonants     : 10
Longest common word sequence: "quick brown"
```

Question 2: Object Sorting and the final Keyword [10 Marks]

A company wants to manage and sort its employees. Create a class `Employee` and a driver class `EmployeeSorting` as described below.

Part A: Class design using final (5 marks)

- `Person` is a non-final class with a protected final `String` `name` and a final method `getCompany()` that returns the company name.
- `Employee` must be declared as a final class that extends `Person`. Its data members `empId`, `department`, `salary`, `joiningYear` must be private final and assigned only in the constructor.
- Declare `COMPANY` (`String`) and `BONUS_RATE` (10%) as public static final constants. Write `calculateBonus()` that uses a final local variable.
- Override `toString()` to display the employee details neatly.

Part B: Sorting objects (5 marks)

Store at least 6 employees (two of them must have the same salary) in a final `List<Employee>` and display the list sorted as follows:

Natural ordering: ascending by `empId` by implementing the `Comparable` interface and using `Collections.sort()`.

Salary: descending, using `Comparator.comparingDouble(...).reversed()`.

Department, then Salary (descending), then Name: using `thenComparing()` to chain comparators.

Joining year: oldest first, using an anonymous inner class implementing `Comparator`.

Name (A-Z): convert the list to an array and sort it with your own selection-sort method `sortEmployees(Employee[] arr, final Comparator<Employee> cmp)` called with a lambda expression.

Sample Input / Output:

Company: IIT Bongora Private Limited.

Sorted by Employee ID (Comparable):

ID	Name	Dept	Salary	Year	Bonus
101	Ananya	HR	48000.00	2021	4800.00
102	Ishita	Finance	61000.00	2020	6100.00
108	Kabir	IT	72000.00	2016	7200.00

Sorted by Salary (descending):

ID	Name	Dept	Salary	Year	Bonus
108	Kabir	IT	72000.00	2016	7200.00
102	Ishita	Finance	61000.00	2020	6100.00
101	Ananya	HR	48000.00	2021	4800.00

Sorted by Department, then Salary (desc), then Name:

ID	Name	Dept	Salary	Year	Bonus
102	Ishita	Finance	61000.00	2020	6100.00
101	Ananya	HR	48000.00	2021	4800.00
108	Kabir	IT	72000.00	2016	7200.00

Sorted by Joining Year (anonymous class):

ID	Name	Dept	Salary	Year	Bonus
108	Kabir	IT	72000.00	2016	7200.00
102	Ishita	Finance	61000.00	2020	6100.00
101	Ananya	HR	48000.00	2021	4800.00

Sorted by Name using manual selection sort:

ID	Name	Dept	Salary	Year	Bonus
101	Ananya	HR	48000.00	2021	4800.00
102	Ishita	Finance	61000.00	2020	6100.00
108	Kabir	IT	72000.00	2016	7200.00

Question 3: String Pattern Matching and Custom Exceptions [10 Marks]

Validation rules

- Email ID:** must be alphanumeric, must contain @ followed by a domain name (for example gmail.com, iiitg.ac.in). It must also contain a special character from the set { @, !, #, \$, &, * } in the first part before @.
- PIN code** of the user address must be a 6-digit number.
- Roll number** should start with the prefix "stud" followed by 5 digits.
- Employee id** should start with the prefix "emp" followed by 3 digits.

1) If any validation fails, it should throw an exception. Hence, you need to create **custom exceptions** specific to Email ID, PIN code, Roll number and Employee id. On encountering a particular exception, the console should print the appropriate error message specific to the exception encountered. (10)

Guidelines

- Create a base class ValidationException and four subclasses: InvalidEmailException, InvalidPinCodeException, InvalidRollNumberException, InvalidEmployeeIdException.
- Perform validation using regular expressions (Pattern and Matcher / matches()) inside a Validator class. Each validate method must declare and throw its own exception.
- In main(), read all four values from the user and use a separate try-catch block for each so that every failing field prints its own message.
- Note: the character @ is the separator in an email address, so treat the "special character before @" as one of ! # \$ & * appearing in the first part (state this assumption in a comment).

Sample Input / Output:

```
Enter Email ID : abc1@iiitg.ac.in
Enter PIN code : 781001
Enter Roll number : std12345
Enter Employee id : emp123
EmailException -> Invalid Email ID: ... (no special character before @)
PIN code is valid.
RollNoException -> Invalid Roll number: ... must start with 'stud' followed by 5 digits.
Employee id is valid.
```

*** End of Assignment ***